Restaurant Rush: Code Explanation (SC2002 syllabus-only version)

This guide explains every code file in the three stages. It is written so you can **read the code, understand it, and explain it in your video**. Every Java feature used comes from your lecture notes (Chapters 2 to 10). Section 2 shows exactly which feature comes from which chapter.

1. The three stages and how to run them

Folder	What it adds	Unit checks	Replay runs
`stage1/`	Regular customers, Waiter, Chef, Cashier, the six-step turn, victory/defeat	31	victory, 2 defeats, invalid-input
`stage2/`	VIP and Critic customers (different behaviour via inheritance and overriding)	39	victory, harsh-review victory, Critic walk-out defeat
`stage3/`	Host, ComboMeal, Happy Hour	61	victory, victory without Happy Hour, defeat, Happy Hour demo

Each folder is complete and runnable on its own (needs JDK 17 or newer):

```
cd stage3
./build.sh                                    # Windows: build.bat
./run.sh                                      # play it yourself (Windows: run.bat)
./run.sh --echo < tests/stage3/victory.in     # replay a recorded winning run
tests/run_tests.sh 3                          # unit tests + compare every replay with its
expected output
```

Commands you type (each turn, once for each staff member who is asked): `seat <n>`, `order <n>`, `serve <n>`, `cook <n>`, `wait` (or just Enter), `help`, `quit`. In Stage 3 you are also asked `Happy Hour (...)>`; type `hh` to switch it on or press Enter to skip.

2. Syllabus compliance

2.1 Where each feature comes from

Feature used in the code	Chapter	Where you see it
Classes, objects, constructors, `this`, fields, methods, `static`, `final`, constants	Ch2 Class and Object	everywhere; `Money`, `Customer.START_SATISFACTION`
`private` / `public` / package-private / `protected`, packages, `import`	Ch2	fields are private; package-private methods such as `Customer.seatAt`
Encapsulation, getters	Ch2	all model classes

Feature used in the code	Chapter	Where you see it
Inheritance, `extends`, `super(...)`, abstract classes, abstract methods, `@Override`	Ch3 Inheritance	`Customer` and its subclasses, `StaffTask` and its subclasses, `Staff` and `ManagedStaff`
Exceptions: `try` / `catch`, `throw`, `throws`, custom exception classes, checked and unchecked	Ch4 Exception Handling	`InvalidChoiceException` (checked), `GameRuleException`, `GameQuitException` (unchecked), `NumberFormatException`
`Scanner` for input	Ch4 (and Ch3)	`ConsoleUI`
Polymorphism, overriding, dynamic binding, a base-class reference calling the subclass method	Ch5 Polymorphism	`customer.billFor(...)`, `task.execute(...)`, `item.getPrepUnits()` on a `ComboMeal`
`interface` and `implements`	Ch5 and Ch9	`GameUI` implemented by `ConsoleUI`
UML class and sequence diagrams	Ch6, Ch7	in the report
Model the application (entity, control, boundary classes)	Ch8	packages `model`, `control`, `ui`
Design principles (SRP, OCP, LSP, ISP, DIP)	Ch9	see section 3.3
Generics (`List<String>`), `ArrayList`, enhanced `for`, `Arrays.asList`	Ch10	all over
Lambda/method reference, Stream API (`filter`, `collect(Collectors.toList())`)	Ch10	`Restaurant.getActiveCustomers()`
Enhanced `switch` (arrow labels, `switch` expression on an `enum`)	Ch10	`CustomerType.create`, `Waiter.createTask`, `GameController.chooseTask`
`enum`	Ch2/Ch10	`CustomerState`, `OrderStatus`, `CustomerType`, `CommandKind`

Only these library classes are imported: `java.util.ArrayList`, `java.util.Arrays`, `java.util.List`, `java.util.Scanner`, `java.util.stream.Collectors`. Everything else (string building, padding, number formatting) is plain Java.

2.2 What I changed from the earlier version, and why

An earlier version of this project used some things your notes do not cover. I rewrote it, and the game behaves **exactly the same**: all recorded replay outputs match the earlier outputs character for character.

Earlier version used	Not in the syllabus, so now
`Optional`, `OptionalDouble`	return `null` when there is no result (documented in each method)
`record` classes	ordinary classes with private fields and getters
`String.format`, `printf`, `String.repeat`, `StringBuilder`, `String.join`	`+` concatenation, and small helper methods `pad`, `repeat`, `join`
`BufferedReader`, `PrintStream`	`Scanner` and `System.out`
`LinkedList`, `Map`, `TreeMap`, `LinkedHashMap`	`ArrayList` and small classes (`Arrival`, `RoleOptions`, `SatisfactionGain`)

Earlier version used	Not in the syllabus, so now
`List.of`, `List.copyOf`, `Collections.unmodifiableList`	`Arrays.asList` and copying into a new `ArrayList`
`Comparator`	a plain loop that remembers the best candidate
`IllegalStateException`	our own `GameRuleException`
enum constants with their own method bodies	`switch` expression over the enum
`instanceof` chains in the game logic	a `getVerb()` method (and polymorphism)
`String.trim`, `toLowerCase`, `replace`, `split`, `isEmpty`	`Scanner` over the line, `toUpperCase`, `size() == 0`
controller depending on the concrete console class	controller depends on the `GameUI` **interface** (Ch9 DIP)
validation returning `Optional<String>`	validation throws a custom **checked exception** (Ch4)

3. Design overview

3.1 Packages (Ch8: entity / control / boundary)

- `restaurant.model` is the **entity** layer: the things in the restaurant (`Restaurant`, `Customer`, `Order`, `Table`, `Menu`...). It holds the rules of what is allowed to happen to them.
- `restaurant.staff` are **control objects for actions**: each task (`SeatCustomerTask`...) knows how to check and carry out one action. `Waiter`, `Chef`, `Host` say which tasks they accept.
- `restaurant.control` is the **control** layer: `GameController` runs the six-step turn; `GameConfig` holds the setting; `TurnPlan`, `GameResult`, the exceptions and the `GameUI` interface support it.
- `restaurant.ui` is the **boundary**: `ConsoleUI` reads the keyboard and prints. It contains **no game rules**.

Dependencies point one way: `Main` creates things, `GameController` uses `GameUI` (interface), `ConsoleUI` implements `GameUI`.

3.2 One turn (what `GameController.playTurn` does)

1. **Arrivals**: if the schedule says someone arrives this turn, create the customer (`CustomerType.create`) and add to the queue.
2. **Manager choices**: for each staff member in order (Host, Waiter, Chef) show the board, ask what to do, *validate* the choice. A refused choice prints the reason and asks again; it costs nothing. Valid choices are stored in a `TurnPlan`.
3. **Kitchen and serving**: execute the stored tasks in order (Host, Waiter, Chef). The Waiter acts before the Chef, so a dish finished by the Chef this turn can only be served next turn, while the Chef can cook an order that the Waiter takes this turn.
4. **Eating and payment**: customers served on an earlier turn become READY_TO_PAY; the Cashier collects one payment (from someone who was already READY_TO_PAY before this turn).
5. **Waiting and abandonment**: every customer still waiting for a table or food loses satisfaction (Regular 8, VIP 5, Critic 12). At 0 the customer leaves, the order is cancelled, the table is freed.
6. After the last turn (18) the three targets are checked: at least 4 paid customers, revenue at least $45.00, average satisfaction at least 60.

3.3 Design principles you can cite (Ch9), tied to real code

- **SRP**: `Customer` knows customer behaviour; `Restaurant` changes entities together; each `StaffTask` checks and executes exactly one action; `ConsoleUI` only does input/output.
- **OCP**: Stage 2 added `VIPCustomer` and `CriticCustomer`; Stage 3 added `Host`, `ComboMeal`, `HappyHour`. `GameController`'s turn flow and the four tasks did not need rewriting for the new customer types and the combo (see the change tables).
- **LSP**: a `VIPCustomer` or `CriticCustomer` can be used wherever a `Customer` is expected; a `ComboMeal` wherever a `MenuItem` is expected (a unit test runs all three customer types through the same dining flow).
- **ISP**: `GameUI` has only what the controller needs. (The staff side uses small abstract classes; we do not claim more.)
- **DIP**: the controller holds a `GameUI` reference, never `ConsoleUI`. The high-level control code depends on an abstraction.

---

4. Stage 1: file-by-file

Stage 1 has 37 Java source files plus the test program. They are explained here in dependency order (small building blocks first).

4.1 `model` package

`GameRuleException.java` extends `RuntimeException` (Ch4: custom *unchecked* exception). It is thrown when code tries something the rules never allow (for example, taking the order of a customer who is not seated). It signals a **programming mistake**, so the compiler does not force anyone to catch it. A *wrong choice by the manager* is different: that uses `InvalidChoiceException` (section 4.3).

`CustomerState.java` an `enum` of the seven places a customer can be: `WAITING` (in the queue) -> `SEATED` -> `ORDERED` -> `EATING` -> `READY_TO_PAY` -> `PAID`, or `LEFT` (left unhappy). An enum is used instead of strings or numbers so that the compiler stops typos.

`OrderStatus.java` an `enum`: `PLACED` -> `PREPARING` -> `READY` -> `SERVED` -> `PAID`, or `CANCELLED`.

`Money.java` a `final` utility class (cannot be extended) with a `private` constructor (cannot be created); it only has `static` methods (Ch2).

- All money in the game is stored as whole **cents** (`int`), so there are no floating-point errors.
- `format(int cents)`: `1350` -> `"$13.50"`. It divides by 100 for dollars, takes `% 100` for cents and adds a leading `"0"` if the cents are below 10 (`1805` -> `$18.05`).
- `percentOf(int cents, int percent)`: `(cents * percent + 50) / 100`. Adding 50 before whole-number division rounds half up to the nearest cent (e.g. 90% of 2430 = 2187).

`MenuItem.java` one dish: `name`, `priceCents`, `prepUnits` (how many chef turns it needs). Fields are `private final` (set once in the constructor). The constructor throws `IllegalArgumentException` for a negative price or fewer than 1 unit (Ch4). `toString()` is overridden to return the name. It is a normal non-final class so `ComboMeal` can extend it in Stage 3.

`Menu.java` holds an `ArrayList<MenuItem>`.

- `add(item)` returns `this` so you can chain calls.
- `getOfferings()` returns a **copy** of the list (a defensive copy) so outside code cannot change the menu by accident.

- `static standard()` builds the required base menu: Salad $9 (1 unit), Burger $15 (1), Pasta $18 (2).

`Table.java` one table: `number` and `occupant`. `isFree()` is `occupant == null`. `occupy` and `release` are **package-private** (no modifier): only classes in `restaurant.model` (that is, `Customer`) may seat someone, so nobody outside can put two customers at one table. `occupy` throws `GameRuleException` if the table is taken.

`WaitingQueue.java` an `ArrayList<Customer>` used as a first-come-first-served queue. `join` and `remove` are package-private; `snapshot()` returns a copy for display.

`Order.java` one customer's order: `customer`, `item`, `billCents` (**locked in** when the order is created, so later discounts never change an existing order), `remainingUnits`, `status`.

- The constructor is package-private: only `Restaurant` creates orders.
- `isCookable()` is true while `PLACED` or `PREPARING`.
- `cookOneUnit()` reduces `remainingUnits`; when it reaches 0 the order becomes `READY`, otherwise `PREPARING`.
- `markServed()`, `markPaid()` and `cancel()` change the status and throw `GameRuleException` if called at the wrong time (for example serving an order that is not READY).

`Customer.java` the **abstract** base class (Ch3). It has the state machine and everything common to all customers.

- Fields (all `private`): `id`, `arrivalTurn`, `satisfaction` (starts at 100), `state`, `table`, `order`, `servedTurn`, `readyToPayTurn`, `recordedSatisfaction` (the value stored at payment).
- **Abstract methods** (each subclass must supply them): `getTypeName()` and `getLossPerWaitingTurn()`.
- `chooseItem(Menu)` is the documented food rule: customer number *k* takes the `((k-1) mod n)`-th item on the menu. It is deterministic, so runs can be repeated. (`CriticCustomer` overrides it in Stage 2.)
- Queries: `isWaitingForService()` (WAITING, SEATED or ORDERED: these customers lose satisfaction) and `isActive()` (not PAID and not LEFT).
- State transitions are package-private and each starts with `require(expectedState)`, which throws `GameRuleException` if the customer is not in the expected state. This is **encapsulation**: no caller can skip a step. `seatAt` -> SEATED (and tells the table), `placeOrder` -> ORDERED, `startEating` -> EATING, `finishEating` -> READY_TO_PAY, `pay` -> PAID (records satisfaction, marks the order PAID, frees the table).
- `loseSatisfaction()` subtracts the customer's own loss, never going below 0 (`Math.max`), and returns true when it reaches 0.
- `abandon()` cancels the order, frees the table, sets state LEFT.
- `toString()` gives `#3 Regular`, used in messages.

`RegularCustomer.java` `extends Customer` (Ch3): calls `super(id, arrivalTurn)`; `getTypeName()` returns "Regular"; `getLossPerWaitingTurn()` returns the constant 8. Both methods use `@Override`.

`CustomerType.java` an `enum` that names the kinds of customer in a schedule. Its method `create(id, turn)` uses a **switch expression** on `this` (Ch10 enhanced switch): `case REGULAR -> new RegularCustomer(id, arrivalTurn);`. Returning the base type `Customer` is polymorphism: the caller never needs to know which subclass it got.

`Arrival.java` a tiny class: "on turn T a customer of type X arrives".

`ArrivalSchedule.java` an `ArrayList<Arrival>`. `add(turn, type)` returns `this` (chainable). `typeAt(turn)` loops through the list and returns the type or **`null`** if nobody arrives that turn.

`Restaurant.java` the entity that **owns** the tables, queue, customers and the score. All rule-changing actions go through it so that other classes never touch an object's state directly.

- Fields: `menu`, `tables`, `queue`, `customers` and counters `revenueCents`, `paidCount`, `paidSatisfactionTotal`, `servedCount`, `unhappyDepartures`.
- Constructor creates `tableCount` tables numbered from 1.
- `getActiveCustomers()` uses a **stream with a method reference**: `customers.stream().filter(Customer::isActive).collect(Collectors.toList())` (Ch10).
- `findCustomer(id)` loops and returns the customer or `null`.
- `admit(c)` adds to `customers` and `queue`.
- `seat(c)` finds the lowest-numbered free table, seats the customer, removes from the queue (throws `GameRuleException` if no table is free; the task checks this first).
- `takeOrder(c)`: the customer chooses with `c.chooseItem(menu)`, the price is `quote(c, item)`, an `Order` is created and attached.
- `quote(c, item)`: Stage 1 returns the item price (later stages change only this method).
- `serve(c, turn)`: marks the order served, customer EATING, adds 1 to `servedCount`.
- `finishEating(turn)`: for each customer who is EATING and was served on an **earlier** turn, move to READY_TO_PAY and return who changed.
- `nextToPay(turn)`: finds the READY_TO_PAY customer who has been ready the longest and became ready on an earlier turn (so nobody pays in the same turn they become ready). The loop keeps a `best` variable; because customers are in arrival order and the comparison is strict `<`, ties go to the earlier arrival. Returns `null` if nobody qualifies.
- `collectPayment(c)`: pays, adds the bill to the revenue, counts the payment and adds the recorded satisfaction.
- `applyWaitingLoss()`: for every waiting customer subtract their loss; if it reaches 0 remove from the queue (if still waiting), `abandon()`, count the unhappy departure, and return the customers who left.

4.2 `staff` package

`Staff.java` abstract base class: holds the `role` name ("Waiter") and `getRole()`.

`StaffTask.java` the abstract class for one thing a staff member can be told to do. This is the **Command pattern**: a task is an object that can be *validated now* and *executed later*.

- Fields: `actorRole`, `verb` (the typed word, e.g. "seat"), `target` (the customer).
- `describe()` returns text like `seat 3`.
- **`abstract void validate(restaurant, plan) throws InvalidChoiceException`**: returns normally if allowed, otherwise throws with the reason. It must not change anything.
- **`abstract String execute(restaurant, turn)`**: does it and returns a sentence describing what happened.
- Overridable hooks with default `false`: `claimsTable()`, `createsOrder()`, `conflictsWith(other)`; the `TurnPlan` uses them.

`SeatCustomerTask.java` verb `"seat"`. `validate` refuses if the customer is not WAITING (message includes the current state) or if `freeTableCount() - plan.tablesClaimed() <= 0` ("no free table"). `execute` calls `restaurant.seat(...)` and reports the table. `claimsTable()` returns true. `conflictsWith` is true when another seat task targets the same customer (it compares `other.getVerb()` and `other.getTarget()`; no `instanceof` needed).

`TakeOrderTask.java` verb `"order"`. `validate` requires state SEATED. `execute` calls `restaurant.takeOrder`, then reports the dish the customer chose with its price and units. `createsOrder()` returns true (so the Chef may cook it this same turn).

`ServeOrderTask.java` verb `"serve"`. `validate` refuses if there is no order, or if the order is not READY (the message explains that a dish finished this turn can only be served next turn). `execute` serves and reports.

`CookOrderTask.java` verb `"cook"`. `validate`: if the customer has no order yet it is still allowed when `plan.willHaveOrder(customer)` (the Waiter takes the order this turn); otherwise it refuses with the reason ("no order yet", "already READY", ...). `execute` calls `order.cookOneUnit()` and reports "unit 1 of 2" and "-> READY" when it finished.

`ManagedStaff.java` abstract subclass of `Staff` for people the manager directs (one task each per turn).

- `abstract List<String> getVerbs()` and `abstract StaffTask createTask(verb, target)`: each role says what it accepts and how to build the task.
- `taskFor(verb, target)`: compares the typed word with the known verbs ignoring letter case (`toUpperCase().equals(...)`) and builds the task, or returns `null` for an unknown verb.
- `availableTasks(restaurant, plan)`: for every active customer and verb, build the task and call `validate` inside `try`/`catch (InvalidChoiceException e)`. If no exception, the task is offered. The empty `catch` is deliberate: "refused" simply means "not offered".

`Waiter.java` `getVerbs()` returns `Arrays.asList("seat", "order", "serve")`. `createTask` uses a **switch expression on the String** to build the right task subclass and returns it as `StaffTask` (polymorphism).

`Chef.java` verbs: only `"cook"`.

`Cashier.java` extends `Staff` (the manager does not direct the Cashier). `collectPayment(restaurant, turn)` asks the restaurant for `nextToPay(turn)`; if none returns `null`; otherwise collects the payment and returns the message "Cashier collects $9.00 from #1 Regular (PAID, satisfaction 84 recorded, table freed)".

4.3 `control` package

`InvalidChoiceException.java` `extends Exception` (Ch4: custom **checked** exception). Its message is the reason shown to the manager. Because it is checked, the compiler forces every caller of `validate` to handle it (`try`/`catch` or `throws`), which is exactly what we want: a refused choice must always be handled.

`GameQuitException.java` `extends RuntimeException`; thrown when the manager types `quit` or the input ends. `Main` catches it.

`CommandKind.java` `enum` {TASK, WAIT, HELP, QUIT, INVALID}.

`ManagerCommand.java` a plain data class: `kind`, `verb`, `targetId`, `message`, with getters. It is what the UI hands to the controller after reading a line.

`RoleOptions.java` a data class: role name and the list of tasks available to that role (for display).

`GameUI.java` an **interface** (Ch5, Ch9): `showWelcome`, `showHelp`, `showTurnHeader`, `showEvents`, `showBoard`, `promptStaff`, `showError`, `showTurnSummary`, `showResult` (Stage 3 adds `askHappyHour`). The controller only knows this interface (Dependency Inversion).

`GameResult.java` the final outcome: paid count, revenue, total of recorded satisfaction, whether each of the three targets was met, and `isVictory()` (all three).

`GameConfig.java` the setting: 18 turns, 2 tables, targets (4 paid, $45.00 = 4500 cents, average 60), and the `ArrivalSchedule`. `base()` builds the required setting. `newMenu()` and `newRestaurant()` create fresh objects. `evaluate(restaurant)` checks the targets; the average is tested as `total >= minAverage * paidCount` in whole numbers, so there is no rounding problem.

`TurnPlan.java` the tasks chosen **so far this turn**. Validation needs to know "what will already have happened": `tablesClaimed()` counts tasks that will use a table, `willHaveOrder(customer)` says whether the Waiter will take that customer's order, `findConflict` / `conflicts` detect clashes. Tasks are executed in the order they were added.

`GameController.java` the control class that runs the game (section 3.2).

- Fields: the `config`, the `restaurant`, a `GameUI ui`, the `roster` (`List<ManagedStaff>`: Waiter then Chef), the `cashier`, and `nextCustomerId`.
- `run()` shows the welcome, plays turns 1 to 18 and returns the `GameResult` after `config.evaluate`.
- `playTurn(turn)` runs the steps in order and prints each step's events through `ui.showEvents`.
- `arrivals(turn, events)` asks the schedule for a `CustomerType`; if not `null` it creates the customer (polymorphic factory), admits them and adds an event line.
- `collectChoices()` builds a `TurnPlan` by asking each staff member in turn.
- `chooseTask(staff, plan)` is a `while (true)` loop: show the options, ask the UI, then a `switch` on the command kind: `WAIT` returns `null`; `QUIT` throws `GameQuitException`; `HELP` shows help; `INVALID` shows the error; `TASK` calls `buildTask`, and returns the task if it is valid.
- `buildTask` checks, in order: the customer exists; the staff member understands the verb; `task.validate(...)` (inside `try`/`catch (InvalidChoiceException e)` which shows `e.getMessage()`); no clash with the plan. If any check fails it shows the reason and returns `null`, so the loop asks again, **without using up the staff member's task**.
- `describeAvailable` builds the "Staff tasks available" lists for the board.
- The controller never asks "what kind of customer is this?" or "what kind of task is this?". It calls the overridden methods.

4.4 `ui` package and `Main`

`ConsoleUI.java` `implements GameUI` (the boundary). It has a `Scanner in` and an `echo` flag.

- `readLine(prompt)` prints the prompt, then loops: if there is no more input it throws `GameQuitException`; lines whose first word starts with `#` are comments and are skipped (used in the replay files); with `echo` on, the line is printed so a piped run shows what was typed.
- `parse(raw)` uses a second `Scanner` over the line to split it into words: no words means WAIT; one word is `wait`/`w`, `help`/`?`, `quit`/`exit` or "not understood"; two words are a verb and a number, converted by `Integer.parseInt` inside `try`/`catch (NumberFormatException e)` (Ch4) so "seat abc" gives a friendly message; anything else is "not understood".
- `showBoard` prints the start-of-turn state: turn, revenue, average satisfaction, paid count, dishes served, unhappy departures, the waiting queue, the tables, a table of customers (number, type, state, satisfaction, order with status), and the available tasks.
- `showEvents`, `showError`, `showTurnSummary`, `showResult` print the rest.
- Helpers replace `printf`: `pad(text, width)` adds spaces on the right (like `%-8s`), `repeat(char, n)`, `join(list, separator)`, and `formatAverage(total, count)` which rounds half up to one decimal using whole numbers: `(20*total + count) / (2*count)` gives tenths. It returns `"N/A"` when nobody has paid.

`Main.java` creates a `Scanner` on `System.in`, reads the `--echo` argument, creates the `ConsoleUI`, and runs `new GameController(GameConfig.base(), ui).run()` inside `try`/`catch (GameQuitException e)`.

4.5 Tests and scripts

`src/test/java/restaurant/RuleTests.java` a plain Java program (no JUnit; it is not in the syllabus). `check(name, boolean)` counts passes and failures. Each static method sets up a `Restaurant`, performs actions and checks the rules, for example: a dish finished this turn cannot be served this turn; a customer served on turn 4 becomes READY_TO_PAY on turn 5 and pays on turn 6; invalid choices are refused with a reason; the Chef may cook an order the Waiter takes this turn; Pasta needs two units; a customer at 0 leaves and the table is freed; a served customer loses nothing; skipping a step throws `GameRuleException`. The helper `problem(task, restaurant, plan)` runs `validate` in `try`/`catch` and returns the reason or `null`. It exits with code 1 if anything fails.

`build.sh`, `run.sh`, `build.bat`, `run.bat` compile into `out/` and run `restaurant.Main`.

`tests/run_tests.sh <stage>` compiles, runs `RuleTests`, then runs every `tests/stageN/*.in` file through the game and compares the output with the saved `*.expected` file. `--update` rewrites the expected files.

`tests/stage1/*.in` and `*.expected`: scripted input for the game and its exact expected output. `victory.in` wins with 5 paid customers; `defeat_idle.in` does nothing (everyone leaves); `defeat_neglect.in` serves too slowly; `invalid_input.in` tries wrong commands to show each is explained and refused.

4.6 A worked example (Stage 1, turns 1 to 3)

- **Turn 1**: arrivals creates `#1 Regular` (WAITING). The Waiter is told `seat 1` (valid: WAITING, one free table). Step 3 executes it: table 1, state SEATED. Step 5: satisfaction 100 -> 92.
- **Turn 2**: Waiter `order 1`, Chef `cook 1`. The Chef's task is valid only because the `TurnPlan` knows the order will exist. Step 3: the Waiter takes the order (customer 1 picks item `(1-1) mod 3 = 0`: Salad, $9.00, 1 unit), then the Chef cooks the 1 unit: order READY. Step 5: 92 -> 84.
- **Turn 3**: Waiter `serve 1` (valid: READY at the start of the turn). Step 3: SERVED, customer EATING. Served customers no longer lose satisfaction (stays 84).
- **Turn 4**: step 4: served on turn 3, which is earlier, so READY_TO_PAY. Turn 5: Cashier collects $9.00; satisfaction 84 recorded; table freed.

---

5. Stage 2: what is new or changed

New behaviour: two more customer types. Only these files differ from Stage 1.

File	Change
`model/VIPCustomer.java`	**new**
`model/CriticCustomer.java`	**new**
`model/Customer.java`	two new overridable methods
`model/CustomerType.java`	two new enum values and two new `case` lines
`model/Restaurant.java`	`quote` now asks the customer

File	Change
`staff/Cashier.java`	message shows a review adjustment
`control/GameConfig.java`	schedule uses VIP (turn 4) and Critic (turn 10)
`tests/`	new transcripts; 8 more unit checks

`VIPCustomer.java` `extends Customer`. Loses **5** per waiting turn (constant `LOSS_PER_TURN`). Overrides `billFor(item)` to return `Money.percentOf(price, 90)`: 10% off the bill. Example: Burger $15.00 becomes $13.50.

`CriticCustomer.java` `extends Customer`. Loses **12** per waiting turn. Two special behaviours, both by **overriding**:

1. `chooseItem(menu)`: loops through the menu and picks the item with the most preparation units (the first one if tied). In Stage 2 that is Pasta; in Stage 3 it becomes the 3-unit combo automatically, because `ComboMeal` has more units.
2. `satisfactionAtPayment()`: the review written when paying. If satisfaction is at least 60 the review is glowing (+10, at most 100); otherwise it is harsh (-10, at least 0). The value returned is what is recorded and counts in the average.

`Customer.java` (changed) gains two methods with sensible defaults, so `RegularCustomer` needs no change:

- `public int billFor(MenuItem item)` returns the full price.
- `protected int satisfactionAtPayment()` returns the current satisfaction; `pay()` now stores `satisfactionAtPayment()` instead of `satisfaction`. `protected` (Ch2/3) means only the class and its subclasses can call it.

`CustomerType.java` (changed): `REGULAR, VIP, CRITIC;` and the switch expression gets two more arrow cases. (A switch expression over an enum must cover every constant, so the compiler reminds you if you forget one.)

`Restaurant.quote`: `return c.billFor(item);`. This is **polymorphism** (Ch5): the same call produces a full price for a Regular and a discounted price for a VIP, chosen at run time by the actual object type. No `if (customer is VIP)` anywhere.

`Cashier.java`: remembers the satisfaction before payment and, if the recorded value differs, adds `[review adjusted satisfaction 52 -> 42]` to the message.

`GameConfig.base()`: same six arrival turns, but turn 4 is `CustomerType.VIP` and turn 10 is `CustomerType.CRITIC`.

Why this shows good design: `GameController`, all four tasks, `Waiter`, `Chef`, the UI and `TurnPlan` are **unchanged**. New customer types were added by writing subclasses and overriding methods (Open/Closed, Ch9).

---

6. Stage 3: what is new or changed

New behaviour: a Host, a combo meal, Happy Hour.

File	Change
`staff/Host.java`	**new**

File	Change
`model/ComboMeal.java`	**new**
`model/HappyHour.java`	**new**
`model/SatisfactionGain.java`	**new**
`model/Customer.java`	new `recoverSatisfaction`
`model/Restaurant.java`	owns a `HappyHour`; `quote` includes it; recovery; `endTurn`
`control/GameConfig.java`	`newMenu()` adds the combo
`control/GameController.java`	Host in roster; Happy Hour offer and notices
`control/GameUI.java`, `ui/ConsoleUI.java`	`askHappyHour`, help text
`tests/`	new transcripts; 22 more unit checks

`Host.java` `extends ManagedStaff`; its only verb is `"seat"`, and `createTask` returns a `SeatCustomerTask`, **the same task class the Waiter uses**. Because tasks are objects and the roster is a list, adding a new staff member is one new class plus one `roster.add(new Host())` line. The Host is asked first, then the Waiter then the Chef. The `TurnPlan` keeps them from seating the same customer or taking the last table twice (`conflictsWith`, `tablesClaimed`).

`ComboMeal.java` `extends MenuItem` (Ch3). The constructor takes two existing items and calls `super(name, price, units)` with the computed values: the name is "Salad + Pasta Combo", the price is **90% of the sum**, rounded to the cent (`Money.percentOf`), and the preparation units are the **sum** of the two (3). It also keeps the two components (`getFirst`, `getSecond`). Since a `ComboMeal` *is a* `MenuItem`, ordering, cooking and billing code needs no change (LSP and polymorphism).

`HappyHour.java` state: `used` and `remainingTurns`. Constants: 2 turns, bill percent 80 (20% off), recovery 15 points.

- `isAvailable()` (not used yet), `isActive()` (turns left), `getRemainingTurns()`.
- `activate()` sets `used` and turns left to 2; calling it again throws `GameRuleException` (only once per game).
- `endTurn()` counts down once per turn.
- `applyTo(bill)` returns 80% of the bill while active, otherwise the bill.

`SatisfactionGain.java` a data class (customer and points gained) to report who recovered satisfaction; it replaces a `Map` which is outside the syllabus.

`Customer.recoverSatisfaction(points)` raises satisfaction, capped at 100 using `Math.min`, and returns the points actually gained.

`Restaurant` (changed):

- a new field `HappyHour happyHour` with `getHappyHour()`.
- `quote(c, item)` is now `happyHour.applyTo(c.billFor(item))`: first the customer's own rule (VIP discount), then Happy Hour. Example: VIP Burger during Happy Hour: 1500 -> 1350 -> **1080** ($10.80). The result is stored in the `Order`, so the price is **locked at the moment the order is taken**.
- `applyHappyHourRecovery()` gives +15 to every customer still waiting for a table or food and returns the list of `SatisfactionGain`.
- `endTurn()` calls `happyHour.endTurn()`.

`GameConfig.newMenu()` builds the standard menu, takes the Salad and the Pasta from it and adds `new ComboMeal(salad, pasta)` as the fourth item. Customer #4 now orders the combo by the existing rule `((4-1) mod 4) = 3`, and the Critic orders it because it has the most units. No customer code changed.

`GameController` (changed):

- `roster.add(new Host())` before the Waiter.
- In `playTurn`, after the arrivals: if Happy Hour is already active (its second turn) the recovery is applied and reported; then the board is shown with a **Happy Hour notice** ("available (once per game)", "ACTIVE - 2 turn(s) remaining including this one", or "used up"); then `offerHappyHour()` asks `ui.askHappyHour()` if it is still available. If the answer is yes, `activate()` is called and the first recovery is applied and reported.
- `restaurant.endTurn()` runs at the end of every turn, which counts Happy Hour down.

`GameUI` / `ConsoleUI` (changed): the interface gets `boolean askHappyHour()`. In `ConsoleUI` it reads a line, splits it with a `Scanner`, and accepts `hh` / `yes` / `y` (true), Enter/ `skip` / `no` / `n` (false), `quit` (throws `GameQuitException`); anything else shows an error and asks again. `showHelp` lists the new commands.

Tests: new unit checks for the combo price ($24.30 and $21.60) and units, the combo ordering rule, Happy Hour pricing and lock-in, lasting exactly two turns and only once, recovery that is capped at 100 and only for waiting customers, and the Host and Waiter seating two different customers in one turn but refusing the same customer or a third table. The Critic test now expects the combo. Replays: `victory.in` (Happy Hour switched on), `victory_no_happy_hour.in`, `defeat_neglect.in`, and `hh_demo.in` which shows rejected choices and prices locking in.

---

7. Likely questions in the video or viva

- **Where is polymorphism?** `restaurant.quote` calls `c.billFor(item)`; `GameController` calls `task.validate/execute` on `StaffTask` references; `chooseItem` on `Customer`; a `ComboMeal` used as a `MenuItem`.
- **Where is encapsulation?** Private fields everywhere; state changes in `Customer`, `Order`, `Table` are package-private and guarded by `require(...)` so a step cannot be skipped.
- **Why a checked exception for invalid choices?** A wrong choice is an expected, recoverable situation, so the compiler should force every caller to handle it. A bug in the game logic is unexpected, so it is unchecked (`GameRuleException`).
- **Why an interface for the UI?** So the controller does not depend on the console (DIP, ISP); a different UI or a test UI could replace it without changing the controller.
- **Why does the Chef's task validate when the order does not exist yet?** The `TurnPlan` records that the Waiter will take it this turn. Step 3 then runs the Waiter before the Chef.
- **Why keep money in cents?** To avoid floating-point rounding errors; `Money.percentOf` rounds half up to the cent.
- **How is the game reproducible?** The food rule is deterministic; there is no randomness; replay files give the same output every time.